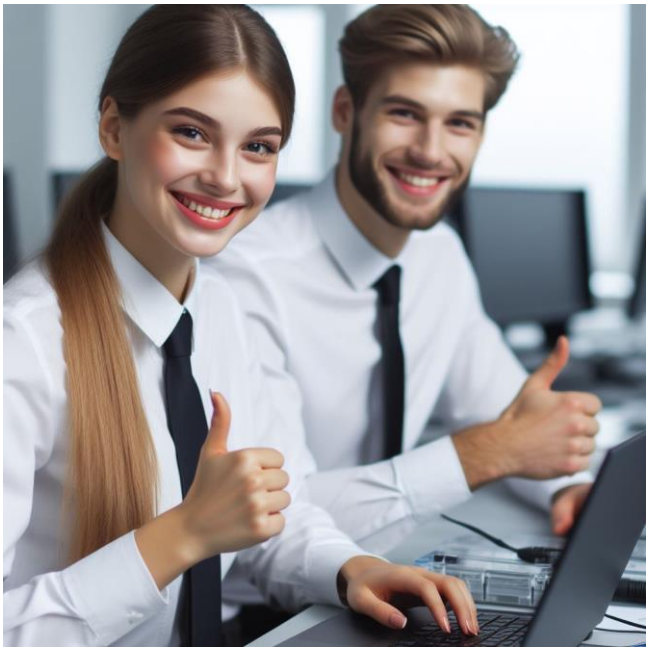
Test Automation with

Cypress

*By Inder P Singh for **SDETs**, **QA** and **Testers***



Download for reference ⬇️ Like 👍 Share 🔗
YouTube Channel: Software and Testing Training (341 Tutorials, 82,000 Subscribers)
Blog: Software Testing Space (1.8 Million Views)

1 Cypress Overview & Architecture 3
2 Installing Cypress & Project Setup 4
3 Cypress End-to-End Testing Fundamentals 6
4 Cypress Test API & Custom Commands 8
5 Cypress Fixtures, Network Stubbing & Test Data Management 10
6 Cypress Component Testing 12
7 Cypress Accessibility Testing 14
8 Cypress UI Coverage & Visual Testing 16
9 Advanced Cypress Patterns & Best Practices 18
10 Cypress Cloud Dashboard & CI/CD Integration 20
11 Cypress Parallelization, Scaling & Flake Reduction 22
12 Cypress Debugging, Logs & Reporting 24
13 Cypress Interview Questions & Answers 26

1 Cypress Overview & Architecture

Q: What is Cypress and how does it differ from other test automation tools?
A: **Cypress** is an end-to-end testing framework for web applications, built entirely in JavaScript. Unlike traditional tools (e.g. Selenium-based), Cypress runs directly in the browser alongside the application under test. It automates at the network and DOM level without using WebDriver, giving faster, more reliable feedback loops. Cypress combines the test runner, assertion library, and utilities into one integrated **Cypress automation** toolset, so there's no need to install multiple libraries or drivers.

Q: How does Cypress architecture work?
A: Cypress consists of two main parts: a Node.js process (the test runner) and a browser client (where tests execute). When you run a Cypress test, a background Node server spins up and launches an Electron or Chrome browser with a special context. The test code is injected into the browser's JavaScript environment, running in the same event loop as the application. This means Cypress has native access to the DOM, window, and application code, allowing it to automatically wait on elements, modify network traffic, and control time. The Node server communicates with the browser to perform privileged tasks (like file system operations, screenshots, and video recording). This architecture (no network detours for commands) is why Cypress commands are fast and provide real-time reactivity.

Q: What are the main components of Cypress?
A: Key components include the **Cypress Test Runner** (a desktop GUI and CLI for running tests), the **Browser Agent** (an injected script that issues commands), and the **Cypress Node Server** (handles tasks like launching browsers, reading configuration, and managing artifacts). Tests are written in familiar Mocha-style `describe/it` blocks and run in a Chrome-based browser. Cypress also provides features like automatic waiting, retries on assertions, and instant reload on file changes, all due to its tight browser integration.

Follow Inder P Singh (18 years' experience in Test Automation) in LinkedIn to get the new AI-based Testing, Test Automation and QA documents.

2 Installing Cypress & Project Setup

Q: How do I install Cypress in my project and get started?
A: First, make sure Node.js is installed. In your project directory (with a `package.json`), run a package manager command to add Cypress as a dev dependency, for example:
`npm install cypress --save-dev`

This installs Cypress locally. Then initialize the Cypress test environment by running:

`npx cypress open`

This will launch the Cypress Test Runner for the first time. Cypress will generate a default cypress/ folder structure and a configuration file (cypress.config.js). The Test Runner's GUI lets you browse example tests, open the interactive test runner, or switch to headless mode via CLI.

Q: What folder structure and config files does Cypress create?
A: By default, Cypress creates a `cypress/` directory with subfolders:

- **`e2e/`** (formerly `integration/`) – where end-to-end spec files go (e.g. `.cy.js` or `.cy.ts` files).
- **`fixtures/`** – for static data files (JSON, images) used in tests.
- **`support/`** – for reusable configuration and custom commands (see next chapters).
- **`videos/`** and **`screenshots/`** – output folders for test recordings.
  Additionally, a `cypress.config.js` file is generated at the project root. In it, you can configure settings like `baseUrl`, default timeouts, viewport size, and spec patterns. For example:

```
// cypress.config.js
module.exports = {
  e2e: {
    baseUrl: 'http://localhost:3000',
    specPattern: 'cypress/e2e/**/*.cy.js'
  }
}
```

Q: How do I run Cypress tests after setup?

A: You can run tests in interactive mode or headless. To open the interactive GUI, use `npx cypress open` and click a test. For continuous integration or batch runs, use the CLI:

```
npx cypress run
```

This runs all spec files headlessly (default in Electron). You can specify a browser (e.g. `--browser chrome`) or record results to the **Cypress Cloud** using `--record` (requires a dashboard key). Test results (screenshots on failure, video) will be saved in the default folders.

3 Cypress End-to-End Testing Fundamentals

Q: How do I write a basic end-to-end test in Cypress?
A: Cypress tests use a familiar structure. For example:

```
describe('Login Flow', () => {
  it('logs in with valid credentials', () => {
    cy.visit('/login');
    cy.get('input[name="username"]').type('alice');
    cy.get('input[name="password"]').type('s3cret');
    cy.get('button[type="submit"]').click();
    cy.url().should('include', '/dashboard');
    cy.contains('Welcome, alice').should('be.visible');
  });
});
```

In this example, `cy.visit()` loads a page, `cy.get()` finds elements, and assertions like `.should()` verify outcomes. Cypress commands (`cy.*`) are asynchronous but chainable; there's no need for `await` or sleeps because Cypress automatically waits for actions and assertions to complete or time out.

Q: What are common Cypress commands for interacting with the page?
A: Common commands include:

- `cy.visit(url)` – navigate to a URL (appends to `baseUrl` if set).
- `cy.get(selector)` – query DOM elements using CSS selectors.
- `cy.contains(text)` – find elements with matching text.
- `cy.type(text)` – type into an input element.
- `cy.click()` – click a button or link.
- `cy.select()` – choose an option in a `<select>` element.
- `cy.url()`, `cy.location()` – get the current URL or location object for assertions.
- `cy.reload()`, `cy.go()` – reload the page or navigate browser history.

These can be chained with assertions (`.should()`, `.and()`) for conditions like visibility, text content, or element attributes. Because of Cypress's retry logic, commands automatically wait until an element appears and until assertions pass, up to the configured timeout.

Q: How does Cypress handle asynchronous waiting?

A: Cypress commands are queued and executed sequentially in the browser. There's no need to write `async/await` or manual delays. For example, `cy.get('.item').should('have.length', 5)` will retry until 5 items exist or the default timeout (4 seconds). If a test step depends on network data, you can use `cy.intercept()` (see next chapter) or use `cy.wait()` on an alias. In general, avoid arbitrary waits like `cy.wait(2000)`; rely on command chaining (`.should()`, `.then()`) to proceed when conditions are met. This results in more stable tests.

Q: How do I organize test code with Mocha syntax?

A: Cypress uses Mocha's BDD (`describe`, `context`, `it`, `beforeEach`, etc.). Use `describe()` or `context()` to group related tests, `it()` for individual test cases, and hooks (`before`, `beforeEach`, `after`, `afterEach`) for setup/teardown. Example:

```
describe('Shopping Cart', () => {
  beforeEach(() => {
    cy.visit('/');
    // maybe reset database or session
  });
  it('adds an item to cart', () => {
    cy.get('[data-cy=add-to-cart]').click();
    cy.get('[data-cy=cart-count]').should('contain', '1');
  });
  it('removes an item from cart', () => {
    // assume cart already has one item
    cy.get('[data-cy=remove-from-cart]').click();
    cy.get('[data-cy=cart-count]').should('contain', '0');
  });
});
```

You should keep tests independent. Don't rely on one test's side effects for another. Use programmatic setup (API calls or fixtures) to isolate state when needed. Avoid hard-coded waits; let Cypress handle timing.

4 Cypress Test API & Custom Commands

Q: What is the Cypress Test API, and how do I write custom commands?
A: Cypress offers a rich, chainable API through the global `cy` object. Common actions (`cy.click`, `cy.type`, `cy.request`, etc.) and assertions (`.should`, `.and`) are available out-of-the-box. To extend this, you can define custom commands in `cypress/support/commands.js` (or `.ts`). For example, to create a login helper:

```
// cypress/support/commands.js

Cypress.Commands.add('login', (username, password) => {

  cy.visit('/login');

  cy.get('#username').type(username);

  cy.get('#password').type(password);

  cy.get('button[type="submit"]').click();

});
```

Now in tests you can call `cy.login('inder', 'password')` as a single command. Use custom commands to encapsulate repetitive flows or to create higher-level APIs for your application.

Q: How do I chain commands and work with return values?
A: Cypress commands are asynchronous and yield "subjects". You can chain commands:

```
cy.get('.item').find('button').click();
```

If you need to work with the yielded value (e.g. element text), use `.then()`:

```
cy.get('.price').invoke('text').then((priceText) => {
  const price = parseFloat(priceText.replace('$', ''));
  expect(price).to.be.lessThan(100);
});
```

You can also alias results with `.as('aliasName')` and refer to them later via `cy.get('@aliasName')`. Avoid mixing synchronous code (like storing a `cy.get()` result in a variable outside a `.then()`), since Cypress commands don't return values directly. Instead use closures or aliases.

Q: How do I perform network requests and stubbing in the Cypress API?
A: The Cypress API includes `cy.request()` for issuing HTTP calls outside the browser (useful for seeding data or testing APIs) and `cy.intercept()` to spy/stub in-browser requests. With `cy.intercept(method, url, response)` you can intercept XHR/fetch calls. For example:

```
// stub a GET request with fixture data

cy.intercept('GET', '/api/items', { fixture:
'items.json' }).as('getItems');

cy.visit('/items');

cy.wait('@getItems').its('response.statusCode').should('eq', 200);
```

`cy.intercept` returns an `alias` you can `cy.wait()` on to ensure the request occurred. It also allows dynamic mocks (e.g. using a function to modify the response). Cypress also supports `cy.route` in older versions, but `cy.intercept` is the modern API.

Q: What are "tasks" in Cypress?
A: Cypress has the concept of tasks (`cy.task()`) to run code in the Node environment (outside the browser). You configure tasks in `cypress.config.js` under `setupNodeEvents(on, config)`. For example, a task can reset a test database or read a file. In a test, you'd call `cy.task('resetDatabase')`, which invokes the code in Node land. Tasks connect the Cypress's browser context and the underlying system, useful for non-UI operations.

5 Cypress Fixtures, Network Stubbing & Test Data Management

Q: What are fixtures in Cypress and how do I use them?
A: Fixtures are static files (typically JSON) stored in the `cypress/fixtures/` directory that you can load in tests as test data. For example, if you have `cypress/fixtures/user.json`:

```
{ "username": "inder", "password": "singh" }
```

In a test you can load it with:

```
cy.fixture('user.json').then((user) => {
  cy.get('#username').type(user.username);
  cy.get('#password').type(user.password);
});
```

Using fixtures keeps test data separate from code, makes tests easier to read, and allows reusing sample data across tests.

Note: To expand your professional network, you're welcome to connect with me (Inder P Singh) in LinkedIn at https://www.linkedin.com/in/inderpsingh/

Q: How do I stub network requests with fixtures?
A: Use `cy.intercept` combined with the `fixture` shorthand. For example:

```
cy.intercept('POST', '/api/login', { fixture: 'login-success.json' }).as('loginRequest');
cy.visit('/login');
cy.get('button[type=submit]').click();
cy.wait('@loginRequest').its('response.statusCode').should('eq', 200);
```

This intercepts the `/api/login` POST and responds with the contents of `login-success.json` from `cypress/fixtures`. You can simulate error responses by customizing status codes and body:

```
cy.intercept('POST', '/api/login', {
  statusCode: 401,
  body: { error: 'Invalid credentials' }
}).as('loginFail');
```

Q: How do I manage dynamic test data?

A: For dynamic data, you can generate it on the fly in tests or via external APIs. One approach is to programmatically create data before a test run (e.g. using `cy.request()` to hit a test-only endpoint or using `cy.task()` to call a script). You can also use environment variables (Cypress supports CYPRESS_* vars or `cypress.env.json`) to store credentials or URLs. When tests need varying data, you might mix fixtures with random generation, e.g.

```
const randomUser = { username: `user${Date.now()}`, email:
`u${Date.now()}@example.com` };
cy.request('POST', '/api/users', randomUser).then((resp) => {
  cy.wrap(resp.body.id).as('newUserId');
});
```

Using `cy.wrap` and `as` we store values for later steps. Test data management allows tests to be independent of each other and to be run in parallel or isolated environments.

6 Cypress Component Testing

Q: What is Cypress component testing and why use it?

A: Cypress component testing allows testing individual UI components (React, Vue, Angular, etc.) in isolation, like unit tests but using the Cypress platform. Instead of launching a full web app, component tests **mount** a single component in a test runner DOM, letting you interact with and assert on that component's behavior. This is useful for catching UI bugs early, testing complex components with isolated state, and getting feedback on changes without spinning up the whole app.

Q: How do I set up Cypress for component testing?

A: After installing Cypress, run `npx cypress open` and choose the "Component Testing" option. This will guide you through installing any required adapters (e.g. `@cypress/react` for React). It will configure a `cypress.config.js` with a `component` section. Ensure your bundler (like Webpack or Vite) is compatible. You place component test files (with `.cy.js` suffix) in your project (e.g. `cypress/component`). Cypress will build and serve the component code, then open the component test runner.

Q: How do I write a simple component test?

A: Use the `cy.mount()` command provided by the component testing framework. For example, testing a React `Button` component:

```
import { Button } from './Button';

it('shows click count', () => {
  cy.mount(<Button label="Click me" />);
  cy.get('button').click();
  cy.contains('Clicked 1 times').should('exist');
});
```

This code mounts the `<Button>` component, simulates a click, and asserts the output text. For Angular or Vue, similar mounting APIs exist (`cy.mount(Component)` or using wrapper components). Component testing focuses on verifying the component's template, event handling, and state, independent of the larger app.

Q: Can I use the same Cypress APIs in component tests?

A: Yes. Inside a component test, you can still use all `cy.get`, `cy.contains`,

`cy.intercept`, etc., as if it were an end-to-end test. You can also use test fixtures or mock props. The main difference is that the browser is showing only the single component in isolation. You can mix interaction commands (clicking, typing) and assertions just like in E2E tests. Component tests run faster since they don't load unnecessary pages, making quick feedback loops.

7 Cypress Accessibility Testing

Q: Why should I include accessibility testing in my Cypress suite?
A: Accessibility (a11y) testing finds out if your app works for users with disabilities, complying with WCAG standards (e.g. proper semantics, ARIA attributes, color contrast). Cypress can automate a11y checks so that issues are caught early in development. Automated checks are not a replacement for manual audits, but they catch many common problems (missing alt text, improper roles, etc.) and can be integrated into your CI pipeline.

Q: What tools does Cypress use for accessibility checks?
A: The common approach is using the axe-core library via the `cypress-axe` plugin. After installing (`npm install cypress-axe axe-core`), you add `import 'cypress-axe'` in `cypress/support/commands.js`. This exposes commands `cy.injectAxe()` and `cy.checkA11y()`. You can also configure rulesets to ignore or only check certain criteria. There are other community plugins, but axe is widely used for automated scans.

Q: How do I write an accessibility test in Cypress?
A: Typically, after navigating to a page or loading a component, you inject the axe runtime and then run checks. For example:

```
cy.visit('/home');

cy.injectAxe();

cy.checkA11y();
```

This will scan the full page and fail the test if any violations are found. You can also target specific areas:

```
cy.get('#main-content').injectAxe().checkA11y();
```

Or exclude elements: `cy.checkA11y({ exclude: [['.skip-a11y']] });`

The test output (in the Cypress runner) shows any violations with rule IDs and nodes, and links to documentation for fixes. Use these reports to remediate issues.

You can follow me, Inder P Singh in LinkedIn for new resources in AI-based software testing and test automation.

Q: What is the difference between automated scans and manual accessibility tests?
A: Automated scans (like those done by `cy.checkA11y()`) catch many rule-based issues but cannot verify all aspects of accessibility (e.g. whether alternative text is contextually appropriate, or if keyboard navigation makes sense). Cypress can help automate repetitive checks and enforce standards, but manual review and user testing (with screen readers, keyboard-only navigation) are still important for complete coverage. In short: use Cypress a11y tests to catch low-hanging fruit (WCAG violations), and complement them with manual QA for realistic user experience.

8 Cypress UI Coverage & Visual Testing

Q: What is UI coverage in Cypress?

A: UI Coverage is a recent feature available via **Cypress Cloud** (the Cypress Dashboard) that maps which UI elements and pages your tests interact with. It automatically generates a visual report of tested vs untested interactive elements (buttons, links, inputs, etc.) across your application. By uploading test runs to Cypress Cloud (with Test Replay enabled), you can see an overlay highlighting which parts of the UI are covered by your tests. This helps identify gaps and redundant tests. (Note: UI Coverage requires the Cypress Cloud service and is currently a premium feature.)

Q: How do I do visual (UI) testing with Cypress?

A: Visual testing means checking that the UI has not regressed visually. Cypress provides basic screenshot capabilities, which you can use to create visual tests by comparing images. For example, you can use `cy.screenshot()` after navigating to a state:

```
cy.visit('/dashboard');

cy.get('#chart').screenshot('dashboard-chart');
```

This saves a screenshot in `cypress/screenshots`. To do an actual visual diff, you'd typically use a third-party library (e.g. an image snapshot plugin) or a dedicated visual testing tool. The general pattern is to capture a baseline image and then compare new runs to it, failing the test if the difference is too large. Cypress Cloud also integrates visual testing: you can upload screenshots or record runs and use external services for image comparisons.

Q: What is visual regression testing?

A: Visual regression testing is the practice of automatically detecting unintended UI changes. When there are code changes, your tests capture screenshots of pages or components. An automated diff compares these to reference images stored from a previous good state. If differences exceed a tolerance, the test fails, alerting you that something changed in the UI (e.g. layout shift, color change). This catches styling bugs that functional tests might miss. In Cypress, you can implement visual regression with community plugins (like snapshot testing libraries) that wrap `cy.screenshot()` and handle comparisons or use cloud services that integrate with test runs to track visual diffs over time.

9 Advanced Cypress Patterns & Best Practices

Q: What are the advanced patterns for writing Cypress tests?
A: Beyond basics, advanced patterns can improve maintainability and performance:

- **Custom Commands & Page Objects:** Use `Cypress.Commands.add` to encapsulate repeated actions (e.g. login, form fill). While the classic Page Object Model (POM) is not the favored pattern in Cypress (because Cypress encourages direct access to elements), you can create classes or modules that return Cypress chains for shared behaviors. Keep these thin, meaning don't hide Cypress commands behind too much abstraction.
- **Aliases & `.then()`:** Store values or elements with `.as()` and retrieve with `cy.get('@alias')`. Use `.then()` to work with jQuery objects or promises. For example, `cy.get('.item').as('itemList')` lets you reuse the alias multiple times.
- **Conditional Testing:** You can use `cy.url().should('include', 'something')` or conditional logic inside `.then()` to skip or adjust tests based on application state.
- **Retries & Timeouts:** Configure global or per-test retry attempts for flake reduction (`retries` in `cypress.config.js`). Use `.should()` and assertions (which retry) instead of manual waits. For long-loading elements, adjust `timeout` options or split steps.
- **Network Stubbing:** Use `cy.intercept` to stub complex backend interactions. For example, simulate slow APIs (`cy.intercept(..., { delayMs: 2000, body: [...] })`) or inject faults (`statusCode: 500`) to test error handling.
- **Fixtures & Test Data:** Keep fixtures lightweight. For large data setup, use `cy.request()` or `cy.task()` to reset DB or seed data.

Q: What are teh best practices for test reliability and clarity?
A: Many best practices for Cypress focus on stability and readability:

- **Selectors:** Use stable, deterministic selectors (e.g. `data-cy` or `data-test` attributes) instead of CSS classes or IDs that may change. For example, `<button data-cy="submit-order">Submit</button>` makes tests clearer and less brittle: `cy.get('[data-cy=submit-order]')`.
- **Test Isolation:** Each test should set up its own state (or share a setup from `beforeEach`). Avoid dependencies between tests. Clean up state after runs (if tests create data, tear it down).

- **Programmatic Authentication:** Instead of using the UI to log in for every test, use `cy.request()` or `cy.task()` to authenticate behind the scenes and set tokens or cookies. This speeds up tests and isolates UI verification.
- **Avoid `cy.wait` Timeouts:** Rely on `cy.intercept` with `cy.wait('@alias')` or assertions to proceed. Only use `cy.wait()` with a route alias, not hard-coded delays (e.g. avoid `cy.wait(3000)`). This reduces flakiness and speeds up tests.
- **Configuration:** Set a `baseUrl` so you can write `cy.visit('/')` instead of full URLs. Use environment variables (`Cypress.env()`) for secrets or dynamic data (never hard-code credentials).
- **Hooks:** Use `beforeEach`/`afterEach` to reuse setup and cleanup. For example, clear local storage or cookies in `beforeEach` to ensure a clean slate.
- **Clear Assertions:** Prefer multiple `.should()` calls instead of one big assertion: `cy.get('#name').should('be.visible').and('have.value', 'John Doe');`
- **CI Friendliness:** Configure reporters (like JUnit or JSON) for CI and test summaries. Capture videos/screenshots on failures (`video: true` in `cypress.config.js`).

Following these patterns makes your **Cypress automation** suite maintainable. Always keep tests readable (clear intent) and deterministic (minimize randomness, use mocks for external services). The official Cypress docs are guides for best practices.

10 Cypress Cloud Dashboard & CI/CD Integration

Q: What is Cypress Cloud (Dashboard) and why use it?
A: Cypress Cloud is a hosted analytics platform for Cypress test runs. When you run `cypress run --record`, results (test names, durations, pass/fail, logs, screenshots, and videos) are uploaded to your Cypress Cloud account. Benefits include a web UI to view test run history, compare runs, and spot flaky tests. The Dashboard offers features like **Test Replay** (step-by-step playback of a failed test) and insights into test performance. It also enables parallel test execution (distributing specs across machines) and automatic grouping of tests by tags or commit.

Q: How do I set up Cypress Cloud in CI pipelines?
A: In your CI configuration (Jenkins, GitHub Actions, CircleCI, etc.), ensure you have:

1. Cypress installed (`npm ci` or `npm install`).
2. A way to run browsers (usually on a VM or container).
3. The Cypress record key (stored securely as an environment variable or secret, e.g. `CYPRESS_RECORD_KEY`).
4. A `cypress run --record --key $CYPRESS_RECORD_KEY` command.

For example, a simplified GitHub Actions step might look like:

```
- name: Run Cypress Tests
  run: |
    npm ci
    npx cypress run --record --key ${{ secrets.CYPRESS_RECORD_KEY }}
```

This will execute all tests headlessly and record them. After the run, you can view results on Cypress Cloud. The Dashboard will show test results with details and artifacts. You can also use `--parallel` (with a `--ci-build-id`) to leverage parallelization (distribute tests among multiple CI workers), and `--group` to distinguish test suites.

Q: How do I integrate Cypress with CI/CD systems?
A: Nearly any CI environment can run Cypress since it's just Node. The steps include:

- Install dependencies (Node, Cypress binary).

- Start your application (if needed) on `localhost` or a test URL (sometimes in `before_script`).
- Run `npx cypress run` (or `npm test` if scripted) after the app is up.
- Archive results: configure your CI to save `cypress/videos/*.mp4` and `cypress/screenshots/` as artifacts.
- Optionally report test results in CI: use a reporter (JUnit XML) so the CI can parse pass/fail stats.
  Most CI providers have example pipelines for Cypress. The important part is ensuring the app and tests share a network (e.g. run the app in a background server) and that environment variables (like CYPRESS_BASE_URL) are set.

Q: How does Cypress Cloud improve CI test automation?
A: With the Dashboard, you gain parallelization and grouping. Instead of a single long test run, you can shard tests across multiple agents:

```
npx cypress run --record --parallel --ci-build-id $BUILD_ID
```

This speeds up suites by running different spec files concurrently. Dashboard also detects flaky tests: if a test intermittently fails, you can see rerun history. It provides email notifications, Slack integration, and trend analytics (average test duration over time). In short, Cypress Cloud makes large-scale **Cypress automation** feasible and observable in continuous integration.

If you found my document useful, please Like and Share my post. If you need to learn from my video tutorials, you can view the top tutorials in my YouTube channel, Software and Testing Training.

11 Cypress Parallelization, Scaling & Flake Reduction

Q: How can I run Cypress tests in parallel to scale my test suite?
A: To parallelize, you must record runs to Cypress Cloud. Use the `--parallel` flag and provide a unique `--ci-build-id` for the pipeline (this groups parallel instances of the same run). For example:

```
npx cypress run --record --parallel --ci-build-id $CIRCLE_WORKFLOW_ID
```

Each CI worker (or agent) will pick a subset of spec files to run. Cypress automatically balances load by tracking durations. This can reduce total time greatly for large suites. Without Dashboard, you can manually split spec files by passing `--spec` lists to different jobs, but Dashboard's built-in parallelism is more convenient.

Q: What strategies reduce flakiness and stabilize tests?
A: Test flakiness often comes from timing issues or shared state. To reduce flakes:

- Use `cy.intercept` and `cy.wait('@alias')` for network responses rather than arbitrary `cy.wait(ms)`.
- Avoid fixed delays; rely on retries. For example, `cy.get(selector, { timeout: 10000 }).should('be.visible')` waits up to 10s.
- Ensure your app is reset between tests: clear cookies, local storage, and reset any backend state. Use `beforeEach` to visit a base URL or reset state via API.
- Configure retries: in `cypress.config.js`, you can add:

```
module.exports = {
  retries: {
    runMode: 2,  // re-run failing tests up to 2 times in CI
    openMode: 0
  }
}
```

- Use stable selectors (`data-*` attributes) and avoid targeting dynamic content or animations prematurely.
- If using random data, use a fixed seed or ensure data is predictable (fixtures or a seed script).
- Split very long tests into smaller steps; short tests fail more clearly.

- For timing issues, you can use `cy.clock()` and `cy.tick()` to control timers in the app during tests.

  Following these practices results in more reliable automation and fewer false failures.

Q: How to scale tests across multiple machines?

A: Besides Dashboard parallelization, you can horizontally scale by distributing different spec sets manually. For example, in GitHub Actions, you could use a matrix job or multiple agents each running a subset of specs. Tools like `npm cypress run --spec 'cypress/e2e/folder1/*'` can target specific directories. In containerized environments (Docker), spawn multiple containers with unique `CYPRESS_RECORD_KEY`s or passing the `--ci-build-id` for Dashboard. Also ensure your application under test can handle concurrent test runs (e.g. by using separate test databases or resetting state in `beforeEach`).

12 Cypress Debugging, Logs & Reporting

Q: How do I debug tests that are failing?

A: Cypress provides interactive debugging tools. In the interactive Test Runner (via `cypress open`), you can click steps to see snapshots of the DOM at each command. Use `cy.pause()` in a test to halt execution mid-test and examine state in the browser. The command `cy.debug()` logs extra information to the console. You can also insert `debugger` statements in your code (inside a `.then()`) to break when devtools are open. Additionally, Cypress automatically prints detailed error messages and stack traces in the console when assertions fail, and clicking on them shows the relevant code.

Q: How can I log information during tests?

A: Use `cy.log('message')` to print custom messages in the Cypress command log. For example, `cy.log('Logging in user: ' + username)`. This appears in the test runner's UI and helps trace test flow. You can also `console.log()` inside a `.then()` or in custom commands; these appear in the browser's DevTools console. For advanced logging (e.g. in CI output), you might use `cy.task('log', message)` where the task writes to stdout or a file. The test reports will include any failed assertions or errors by default.

Q: How do I generate test reports and artifacts?

A: Cypress can output test results via reporters. By default, it prints a nice summary to the console. For CI, you can configure reporters like JUnit or Mochawesome. For example, install and set Mochawesome reporter in `package.json` or pass CLI flags:

```
npx cypress run --reporter junit --reporter-options
"mochaFile=results/results.xml"
```

This produces a JUnit-style XML (`results.xml`) that CI tools can parse for badges or test summaries. Screenshots (`cypress/screenshots/`) and videos (`cypress/videos/`) are saved automatically on failures (configurable via `screenshotOnRunFailure` and `video` settings). You should archive these as artifacts in CI for later analysis. Cypress Cloud also captures these if you record runs.

Q: How do I interpret Cypress test logs?

A: In the interactive runner, each command's result is logged in sequence. In headless mode, logs go to stdout. Each failed assertion shows the actual vs expected value. For

accessibility failures (see earlier), Cypress prints rule IDs. If running in Cypress Cloud, you get a detailed replay UI. When analyzing logs, focus on the first failure: often it indicates the cause. Use the error messages to locate which `cy.get` or `.should` did not meet its condition.

Q: Can I take custom screenshots or videos in tests?

A: Yes. You can manually call `cy.screenshot('name')` at any point to capture the current viewport. The image is saved under `cypress/screenshots`. You can group screenshots with the test name (e.g. `cy.screenshot()` inside a test context). Cypress automatically records a video of the entire test run when using `cypress run` (headless mode) and if `video: true` (default). You can disable videos if not needed. Videos and screenshots for failures help debug issues after the fact. For example, add `afterEach(() => { if (this.currentTest.state === 'failed') { cy.screenshot('on-failure'); } });` to capture on failure (using a test framework hook).

13 Cypress Interview Questions & Answers

Q: What is Cypress and why might you choose it over other tools like Selenium?
A: Cypress is a modern end-to-end testing framework for web apps, built on JavaScript. It runs tests inside the browser, giving it unique capabilities (native DOM access, automatic waits). Compared to Selenium, Cypress is typically faster, easier to set up (no WebDriver or separate server), and provides real-time reloading and an interactive runner. It uses a single language (JS) and includes assertions and mocking out of the box. However, it currently focuses on single-domain testing and supports only Chrome-family browsers (though support has broadened).

Q: How does Cypress handle asynchronous behavior in tests?
A: Cypress uses a command queue and promise-like chaining under the hood. Most `cy` commands automatically retry until success or timeout. You don't use async/await; instead you chain commands and use callbacks (`.then()`). For example, after clicking a button you can immediately do `cy.get(...).should(...)` without manually waiting, because Cypress waits until the DOM updates. This eliminates many timing issues common in other frameworks.

Q: How can Cypress tests be made deterministic and not flaky?
A: Make tests deterministic by controlling external dependencies: stub network calls with `cy.intercept`, seed known data, and use programmatic authentication. Use stable selectors (`data-cy` attributes). Avoid time-based or random data unless seeded. Clean up state between tests, e.g. with API calls or clearing storage. Configure retries so transient failures rerun. In short: isolate each test, mock unpredictable parts, and use Cypress's built-in waits instead of fixed delays.

Q: How do you test file upload or download in Cypress?
A: For file upload, use `cy.get('input[type=file]').attachFile('path/to/file.png')` if you include a plugin like `cypress-file-upload` (or set the DOM file input's value with a Blob). For downloads, since Cypress can't directly capture file downloads in-browser, a common approach is to trigger the download (e.g. clicking a download link) and then use `cy.readFile()` on the output folder (if configured). Alternatively, use `cy.request()` to hit the file URL via Node and verify the response contents.

Q: Can Cypress test multiple browser windows or tabs?

A: By design, Cypress does not support multiple browser windows or tabs. All tests run within a single browser context. If your application normally opens a new window, you can often bypass it by stubbing `window.open` or by using `cy.visit()` to the new URL directly. The Cypress philosophy is to keep tests in one domain/frame. For interactions that require OAuth or third-party logins, you typically simulate or skip those using backdoor methods (like API login).

Q: How do you handle environment-specific configurations in Cypress?

A: Cypress supports `cypress.env.json` or environment variables. You can store sensitive data (API keys, credentials) in `cypress.env.json` (which should not be committed) and access via `Cypress.env('KEY')`. Alternatively, pass `--env KEY=value` on the CLI or set `process.env.CYPRESS_KEY`. In `cypress.config.js`, you can reference `config.env`. Also, use `baseUrl` to point to staging or production endpoints depending on an environment variable.

Q: What browsers does Cypress support and how do you test across them?

A: Cypress supports Chromium-based browsers (Chrome, Edge, Brave), Firefox, and Electron by default. Safari/WebKit support is in beta (via `webkit` target). To test in a specific browser, use the `--browser` flag, e.g. `npx cypress run --browser firefox`. In CI, you can install and specify different browsers on different runs to cover cross-browser testing. Interactive mode also allows switching browsers via the UI.

Q: How do I debug a failing test in a CI environment where I can't see the interactive runner?

A: Rely on screenshots, videos, and logs. Ensure your CI saves Cypress's `videos/` and `screenshots/` artifacts for failed tests. You can also run `npx cypress run --headed --browser chrome` in CI if you need to see a UI (though headless is default). For logs, use a reporter (like `--reporter mocha-junit`) so you get detailed error messages in the build logs. The combination of error tracebacks, screenshots, and videos usually pinpoints where the test failed. Finally, run the failing spec locally in open mode to step through interactively.

Q: What is the difference between `cy.visit()` and `cy.request()`?

A: `cy.visit()` loads a page in the Cypress-controlled browser, just like a user navigating in a real browser (including running the app's JavaScript, rendering HTML, etc.). `cy.request()` makes an HTTP request outside the browser (in Node). It's used for API

calls (setup, teardown, direct testing of endpoints) and it doesn't render pages or run JS. Use `cy.request()` when you want to bypass the UI (e.g. to log in via API or check an API response directly) and `cy.visit()` when you want to simulate a user navigating or to begin an end-to-end flow.

Q: What are some limitations of Cypress?

A: As of now, Cypress is limited to testing web applications. It cannot test mobile apps (unless webview-based) or desktop apps. It also has restrictions on multi-domain testing: all `cy.visit()` calls and application routes must share the same top-level domain per test (this is improving with recent cross-origin support experiments). It only runs in Chrome-family/Firefox browsers (though this covers most use cases). Cypress tests are in JavaScript/TypeScript; if you need other languages, you can call out to other tools via `cy.task()`.

Q: How are some questions about Cypress knowledge?

A: You can expect questions on practical usage: writing selectors, handling forms, stubbing network, custom commands, and CI integration. For example:

- How do you set up a test to stub a network response?
- Explain the role of fixtures in Cypress tests.
- How does Cypress automatically wait for elements?
- What strategies do you use to reduce test flakiness?
- How do you organize large test suites in Cypress?

You can discuss architecture (browser vs. Node), configuration (`cypress.config.js`), and cloud features. You should also be able to write a simple test on the spot, e.g. verifying a login or making a network stub.

You can find the answers to all these questions in the previous chapters. It's my pleasure to answer any further questions you may have. Thank you! Inder P Singh